

GSOI 2026 暑期常规赛

第八试题解

Ehundategh & tfbz

Gioush 大队

2026 年 7 月

目录

T1 鸬鹚天 (skycall)

T2 雨霖铃 (rainbell)

T3 金缕曲 (goldknot)

T4 蝶恋花 (flutter)

吐槽时间

Contents

T1 鸛鵒天 (skycall)

T2 雨霖鈴 (rainbell)

T3 金縷曲 (goldknot)

T4 蝶戀花 (flutter)

吐槽時間

数据点 1 ~ 4 20 Pts

- 设第 i 个位置是否存在星辉为 $x_i \in \{0, 1\}$ 。直接枚举全部 2^n 个序列，再检查每个位置周围的星辉数是否等于 a_i 。

数据点 1 ~ 4 20 Pts

- ▶ 设第 i 个位置是否存在星辉为 $x_i \in \{0, 1\}$ 。直接枚举全部 2^n 个序列，再检查每个位置周围的星辉数是否等于 a_i 。
- ▶ 这一档保证 $T \leq 5$ 且 $n \leq 18$ 。每个序列需要 $\mathcal{O}(n)$ 的时间检查，因此总时间复杂度为 $\mathcal{O}(Tn2^n)$ ，空间复杂度为 $\mathcal{O}(n)$ 。

数据点 1 ~ 4 20 Pts

- ▶ 设第 i 个位置是否存在星辉为 $x_i \in \{0, 1\}$ 。直接枚举全部 2^n 个序列，再检查每个位置周围的星辉数是否等于 a_i 。
- ▶ 这一档保证 $T \leq 5$ 且 $n \leq 18$ 。每个序列需要 $\mathcal{O}(n)$ 的时间检查，因此总时间复杂度为 $\mathcal{O}(Tn2^n)$ ，空间复杂度为 $\mathcal{O}(n)$ 。
- ▶ 这一做法的瓶颈是枚举了整个序列，但实际上只要确定最左侧的位置，之后所有位置都会被唯一确定。

数据点 5 ~ 12 60 Pts

- 这一档保证 $a_1 \neq 1$ 。当 $n \geq 2$ 时，第一个位置满足 $a_1 = x_1 + x_2$ ，所以 $a_1 = 0$ 时必有 $x_1 = x_2 = 0$ ， $a_1 = 2$ 时必有 $x_1 = x_2 = 1$ 。

数据点 5 ~ 12 60 Pts

- ▶ 这一档保证 $a_1 \neq 1$ 。当 $n \geq 2$ 时，第一个位置满足 $a_1 = x_1 + x_2$ ，所以 $a_1 = 0$ 时必有 $x_1 = x_2 = 0$ ， $a_1 = 2$ 时必有 $x_1 = x_2 = 1$ 。
- ▶ 因此不再需要枚举整个序列，也不需要尝试两个初值。令 $x_0 = 0$ 、 $x_1 = \frac{a_1}{2}$ ，再由 $x_i = a_{i-1} - x_{i-1} - x_{i-2}$ 依次恢复后续位置即可。

数据点 5 ~ 12 60 Pts

- ▶ 这一档保证 $a_1 \neq 1$ 。当 $n \geq 2$ 时，第一个位置满足 $a_1 = x_1 + x_2$ ，所以 $a_1 = 0$ 时必有 $x_1 = x_2 = 0$ ， $a_1 = 2$ 时必有 $x_1 = x_2 = 1$ 。
- ▶ 因此不再需要枚举整个序列，也不需要尝试两个初值。令 $x_0 = 0$ 、 $x_1 = \frac{a_1}{2}$ ，再由 $x_i = a_{i-1} - x_{i-1} - x_{i-2}$ 依次恢复后续位置即可。
- ▶ 若 $n = 1$ ，直接检查 $a_1 = x_1$ 。这一做法已经得到了正解中的递推过程，最后的瓶颈只剩下 $a_1 = 1$ 时无法直接确定 x_1 。时间复杂度为 $\mathcal{O}(n)$ ，空间复杂度为 $\mathcal{O}(n)$ 。

正解 100 Pts

- 分别考虑 $x_1 = 0$ 与 $x_1 = 1$ 。令边界外的 $x_0 = 0$ ，根据第 $i - 1$ 个位置给出的数量，可以得到

$$x_i = a_{i-1} - x_{i-1} - x_{i-2} \quad (2 \leq i \leq n).$$

正解 100 Pts

- 分别考虑 $x_1 = 0$ 与 $x_1 = 1$ 。令边界外的 $x_0 = 0$ ，根据第 $i - 1$ 个位置给出的数量，可以得到

$$x_i = a_{i-1} - x_{i-1} - x_{i-2} \quad (2 \leq i \leq n).$$

- 若某一步得到的 x_i 不属于 $\{0, 1\}$ ，当前初值一定不合法。递推结束后，还要单独检查 $a_n = x_{n-1} + x_n$ 。

正解 100 Pts

- 分别考虑 $x_1 = 0$ 与 $x_1 = 1$ 。令边界外的 $x_0 = 0$ ，根据第 $i - 1$ 个位置给出的数量，可以得到

$$x_i = a_{i-1} - x_{i-1} - x_{i-2} \quad (2 \leq i \leq n).$$

- 若某一步得到的 x_i 不属于 $\{0, 1\}$ ，当前初值一定不合法。递推结束后，还要单独检查 $a_n = x_{n-1} + x_n$ 。
- 两个初值至多各对应一种方案，将其中合法方案的数量相加即可。时间复杂度为 $\mathcal{O}(n)$ ，空间复杂度为 $\mathcal{O}(n)$ 。

正解 100 Pts

- 递推式直接来自 $a_{i-1} = x_{i-2} + x_{i-1} + x_i$ ，因此任何合法方案都必须按照它被恢复，不会遗漏。

正解 100 Pts

- ▶ 递推式直接来自 $a_{i-1} = x_{i-2} + x_{i-1} + x_i$ ，因此任何合法方案都必须按照它被恢复，不会遗漏。
- ▶ 反过来，只要递推过程中的每个 x_i 均为 0 或 1，并且最后一个位置也通过检查，那么所有位置的条件都已经依次满足。

正解 100 Pts

- ▶ 递推式直接来自 $a_{i-1} = x_{i-2} + x_{i-1} + x_i$ ，因此任何合法方案都必须按照它被恢复，不会遗漏。
- ▶ 反过来，只要递推过程中的每个 x_i 均为 0 或 1，并且最后一个位置也通过检查，那么所有位置的条件都已经依次满足。
- ▶ 特别地， $n = 1$ 时没有递推过程，只需分别检查两个初值是否满足 $a_1 = x_1$ 。

正解 100 Pts

```
1 void Solve(){
2     Ans=0;Tag=1;
3     scanf("%d",&n);
4     for(int i=1;i<=n;i++){
5         scanf("%d",&Line[i]);
6     }
7     Dp[0]=0;Dp[1]=0;
8     for(int i=2;i<=n;i++){
9         if(Line[i-1]-Dp[i-1]-Dp[i-2]>1||Line[i-1]-Dp[i-1]-Dp[i-2]<0){
10             Tag=0; break;}
11         else if(Line[i-1]-Dp[i-1]-Dp[i-2]==1){
12             Dp[i]=1;
13         }
14         else if(Line[i-1]-Dp[i-1]-Dp[i-2]==0){
15             Dp[i]=0;
16         }
17     }
18     if(Line[n]!=Dp[n-1]+Dp[n]) Tag=0;
```


正解 100 Pts

```
1  if(Tag) Ans++;
2  Dp[0]=0;Dp[1]=1; Tag=1;
3  for(int i=2;i<=n;i++){
4      if(Line[i-1]-Dp[i-1]-Dp[i-2]>1||Line[i-1]-Dp[i-1]-Dp[i-2]<0) {
5          Tag=0; break;}
6      else if(Line[i-1]-Dp[i-1]-Dp[i-2]==1){
7          Dp[i]=1;
8      }
9      else if(Line[i-1]-Dp[i-1]-Dp[i-2]==0){
10         Dp[i]=0;
11     }
12 }
```

正解 100 Pts

```

1      if(Line[n] != Dp[n-1] + Dp[n]) Tag=0;
2      if(Tag) Ans++;
3      printf("%d\n", Ans);
4      return;
5  }
6  int main(){
7      int c, T;
8      freopen("skycall.in", "r", stdin);
9      freopen("skycall.out", "w", stdout);
10     scanf("%d%d", &c, &T);
11     while(T-->0) Solve();

```

Contents

T1 鸪天 (skycall)

T2 雨霖铃 (rainbell)

T3 金缕曲 (goldknot)

T4 蝶恋花 (flutter)

吐槽时间

数据点 1 ~ 4 20 Pts

- ▶ 枚举选择的 $n - 1$ 条道路，使用并查集检查它们是否无环且连通，再统计听雨路的数量是否恰为 k 。

数据点 1 ~ 4 20 Pts

- ▶ 枚举选择的 $n - 1$ 条道路，使用并查集检查它们是否无环且连通，再统计听雨路的数量是否恰为 k 。
- ▶ 这一档保证 $T \leq 5$ 、 $n \leq 10$ 且 $m \leq 18$ 。枚举一个道路集合的时间复杂度为 $\mathcal{O}(m\alpha(n))$ ，总时间复杂度为 $\mathcal{O}(T2^m m\alpha(n))$ 。

数据点 1 ~ 4 20 Pts

- ▶ 枚举选择的 $n - 1$ 条道路，使用并查集检查它们是否无环且连通，再统计听雨路的数量是否恰为 k 。
- ▶ 这一档保证 $T \leq 5$ 、 $n \leq 10$ 且 $m \leq 18$ 。枚举一个道路集合的时间复杂度为 $\mathcal{O}(m\alpha(n))$ ，总时间复杂度为 $\mathcal{O}(T2^m m\alpha(n))$ 。
- ▶ 暴力说明了真正需要控制的是生成树中听雨路的数量。接下来考虑这个数量能够取到的范围。

数据点 5 ~ 8 40 Pts

- ▶ 当 $k = 0$ 时，只保留晴光路并运行一次 Kruskal。若能够连接所有花亭，得到的生成树就是答案。

数据点 5 ~ 8 40 Pts

- ▶ 当 $k = 0$ 时，只保留晴光路并运行一次 Kruskal。若能够连接所有花亭，得到的生成树就是答案。
- ▶ 当 $k = n - 1$ 时，只保留听雨路进行相同处理。

数据点 5 ~ 8 40 Pts

- ▶ 当 $k = 0$ 时，只保留晴光路并运行一次 Kruskal。若能够连接所有花亭，得到的生成树就是答案。
- ▶ 当 $k = n - 1$ 时，只保留听雨路进行相同处理。
- ▶ 这一档的启发是：先尽量选某一类道路，就能得到另一类道路在生成树中的最少数量。时间复杂度为 $\mathcal{O}(m\alpha(n))$ ，空间复杂度为 $\mathcal{O}(n + m)$ 。

数据点 9 ~ 12 60 Pts

- ▶ 这一档满足特殊性质 B，也就是所有生成树中听雨路数量的最小值恰为 k 。沿用上一档的想法，先选择晴光路，再用听雨路连接剩余连通块。

数据点 9 ~ 12 60 Pts

- ▶ 这一档满足特殊性质 B，也就是所有生成树中听雨路数量的最小值恰为 k 。沿用上一档的想法，先选择晴光路，再用听雨路连接剩余连通块。
- ▶ 设晴光路形成了 s 个连通块。任何生成树至少需要 $s - 1$ 条听雨路，而上述过程恰好加入 $s - 1 = k$ 条，因此得到的生成树就是答案。

数据点 9 ~ 12 60 Pts

- ▶ 这一档满足特殊性质 B，也就是所有生成树中听雨路数量的最小值恰为 k 。沿用上一档的想法，先选择晴光路，再用听雨路连接剩余连通块。
- ▶ 设晴光路形成了 s 个连通块。任何生成树至少需要 $s - 1$ 条听雨路，而上述过程恰好加入 $s - 1 = k$ 条，因此得到的生成树就是答案。
- ▶ 这一档已经给出了求听雨路数量下界的方法。它的瓶颈是只能处理目标值恰好等于下界的情况。时间复杂度为 $\mathcal{O}(m\alpha(n))$ 。

数据点 13 ~ 16 80 Pts

- 这一档在前面方法的基础上，再处理特殊性质 C。此时所有生成树中听雨路数量的最大值恰为 k ，于是先选择听雨路，再用晴光路补全生成树即可。

数据点 13 ~ 16 80 Pts

- ▶ 这一档在前面方法的基础上，再处理特殊性质 C。此时所有生成树中听雨路数量的最大值恰为 k ，于是先选择听雨路，再用晴光路补全生成树即可。
- ▶ 若只看听雨路形成的连通块，每个连通块内部最多选出一棵生成树。先选听雨路时已经让听雨路数量达到最大值，所以最终数量恰为 k 。

数据点 13 ~ 16 80 Pts

- ▶ 这一档在前面方法的基础上，再处理特殊性质 C。此时所有生成树中听雨路数量的最大值恰为 k ，于是先选择听雨路，再用晴光路补全生成树即可。
- ▶ 若只看听雨路形成的连通块，每个连通块内部最多选出一棵生成树。先选听雨路时已经让听雨路数量达到最大值，所以最终数量恰为 k 。
- ▶ 至此，我们能够构造听雨路数量最少或最多的生成树。正解还需要说明，两个极值之间的每个整数都能够取到，并给出达到指定 k 的构造。时间复杂度为 $\mathcal{O}(m\alpha(n))$ 。

正解 100 Pts

- 第一次 Kruskal 先选择晴光路，再用听雨路补全生成树。设最终必须加入的听雨路数量为 p ，那么 p 是任意生成树中听雨路数量的最小值。

正解 100 Pts

- ▶ 第一次 Kruskal 先选择晴光路，再用听雨路补全生成树。设最终必须加入的听雨路数量为 p ，那么 p 是任意生成树中听雨路数量的最小值。
- ▶ 更具体地，只看晴光路形成的连通块。每加入一条跨越两个连通块的听雨路，连通块数量恰好减少一，所以连接这些连通块至少需要 p 条听雨路。

正解 100 Pts

- ▶ 第一次 Kruskal 先选择晴光路，再用听雨路补全生成树。设最终必须加入的听雨路数量为 p ，那么 p 是任意生成树中听雨路数量的最小值。
- ▶ 更具体地，只看晴光路形成的连通块。每加入一条跨越两个连通块的听雨路，连通块数量恰好减少一，所以连接这些连通块至少需要 p 条听雨路。
- ▶ 同理，先选择听雨路得到的数量 q 是上界。因此答案存在的必要条件为 $p \leq k \leq q$ 。代码不必单独保存 q ，第二次构造无法选满 k 条听雨路时，就等价于 $k > q$ 。

正解 100 Pts

- 保留第一次选出的 p 条听雨路，并将它们预先加入并查集。随后先扫描其余听雨路，直到总数达到 k ，最后再用晴光路补全。

正解 100 Pts

- ▶ 保留第一次选出的 p 条听雨路，并将它们预先加入并查集。随后先扫描其余听雨路，直到总数达到 k ，最后再用晴光路补全。
- ▶ 若一条候选听雨路连接了两个不同连通块，加入它不会形成环。只要仍存在一棵包含当前森林且含 k 条听雨路的生成树，就可以用基交换把这条边换入，因此这一选择不会破坏可行性。

正解 100 Pts

- ▶ 保留第一次选出的 p 条听雨路，并将它们预先加入并查集。随后先扫描其余听雨路，直到总数达到 k ，最后再用晴光路补全。
- ▶ 若一条候选听雨路连接了两个不同连通块，加入它不会形成环。只要仍存在一棵包含当前森林且含 k 条听雨路的生成树，就可以用基交换把这条边换入，因此这一选择不会破坏可行性。
- ▶ 当听雨路数量达到 k 后，剩余部分只需要用晴光路连接。若无法补全，或扫描完听雨路仍不足 k 条，则答案不存在。

正解 100 Pts

- 第一次选出的 p 条听雨路本身是一片森林，可以继续用听雨路扩展到一片极大的听雨路森林，其边数正好为 q 。所以当 $p \leq k \leq q$ 时，第二次扫描一定能够选出恰好 k 条听雨路。

正解 100 Pts

- ▶ 第一次选出的 p 条听雨路本身是一片森林，可以继续用听雨路扩展到一片极大的听雨路森林，其边数正好为 q 。所以当 $p \leq k \leq q$ 时，第二次扫描一定能够选出恰好 k 条听雨路。
- ▶ 这 p 条道路已经连接了晴光路的所有连通块。加入额外听雨路只会继续合并连通块，因此选满 k 条后，再扫描晴光路一定能够补成生成树。

正解 100 Pts

- ▶ 第一次选出的 p 条听雨路本身是一片森林，可以继续用听雨路扩展到一片极大的听雨路森林，其边数正好为 q 。所以当 $p \leq k \leq q$ 时，第二次扫描一定能够选出恰好 k 条听雨路。
- ▶ 这 p 条道路已经连接了晴光路的所有连通块。加入额外听雨路只会继续合并连通块，因此选满 k 条后，再扫描晴光路一定能够补成生成树。
- ▶ 两次排序的时间复杂度为 $\mathcal{O}(m \log m)$ ，并查集操作的时间复杂度为 $\mathcal{O}(m \alpha(n))$ ，空间复杂度为 $\mathcal{O}(n + m)$ 。

正解 100 Pts

```
1 void Kruskal1(){
2     sort(Edge+1,Edge+m+1,cmpb);
3     for(int i=1;i<=n;i++) Fa[i]=i;
4     int Count=0,i;
5     for(i=1;i<=m;i++){
6         if(Edge[i].Type==0) break;
7         if(Find(Edge[i].St)==Find(Edge[i].Ed)) continue;
8         Count++;
9         Merge(Edge[i].St,Edge[i].Ed);
10    }
11    for(i=1;i<=m;i++){
12        if(Find(Edge[i].St)==Find(Edge[i].Ed)) continue;
13        Merge(Edge[i].St,Edge[i].Ed);
14        Ans.push_back(Edge[i]);
15    }
16    Must=n-1-Count;
17 }
```

正解 100 Pts

```
1 bool Kruskal2(){
2     sort(Edge+1,Edge+m+1,cmpa);
3     for(int i=1;i<=n;i++) Fa[i]=i;
4     k-=Must;
5     for(int i=0;i<(int)Ans.size();i++){
6         Merge(Ans[i].Ed,Ans[i].St);
7     }
8     for(int i=1;i<=m;i++){
9         if(Edge[i].Type==0&&(!k)) continue;
10        if(Edge[i].Type==1&&(k)) return false;
11        if(Find(Edge[i].St)==Find(Edge[i].Ed)) continue;
12        Merge(Edge[i].St,Edge[i].Ed);
13        Ans.push_back(Edge[i]);
14        if(Edge[i].Type==0) k--;
15    }
16    return (int)Ans.size()==n-1;
17 }
```

正解 100 Pts

```
1 void Solve(){
2     int In1,In2,In3;
3     Ans.clear();Can=0;Must=0;
4     scanf("%d%d%d",&n,&m,&k);
5     for(int i=1;i<=m;i++){
6         scanf("%d%d%d",&In1,&In2,&In3);
7         Edge[i]={In1,In2,In3};
8         if(In3==0) Can++;
9     }
10    if(!Get_Tog()){
11        puts("no solution");
12        return;
13    }
14    Kruskal1();
15    if(Must>k) {puts("no solution");return;}
16    if(!Kruskal2()){puts("no solution");return;}
17    for(int i=0;i<(int)Ans.size();i++){
```

正解 100 Pts

```
1         printf("%d %d %d\n",Ans[i].St,Ans[i].Ed,Ans[i].Type);
2     }
3     return;
4 }
5 int main(){
6     int c,T;
7     freopen("rainbell.in","r",stdin);
8     freopen("rainbell.out","w",stdout);
9     scanf("%d%d",&c,&T);
10    while(T-->0) Solve();
11    return 0;
12 }
```

Contents

T1 鸪天 (skycall)

T2 雨霖铃 (rainbell)

T3 金缕曲 (goldknot)

T4 蝶恋花 (flutter)

吐槽时间

数据点 1 ~ 4 20 Pts

- ▶ 当道路数量很少时，可以把每条道路已经经过的次数记录在状态中，枚举下一步经过哪一条道路。

数据点 1 ~ 4 20 Pts

- ▶ 当道路数量很少时，可以把每条道路已经经过的次数记录在状态中，枚举下一步经过哪一条道路。
- ▶ 一条初始权值为 w 的道路只会产生 $\mathcal{O}(\sqrt{w})$ 次正收益，因此所有道路都不再产生收益后，不需要继续扩展状态。

数据点 1 ~ 4 20 Pts

- ▶ 当道路数量很少时，可以把每条道路已经经过的次数记录在状态中，枚举下一步经过哪一条道路。
- ▶ 一条初始权值为 w 的道路只会产生 $\mathcal{O}(\sqrt{w})$ 次正收益，因此所有道路都不再产生收益后，不需要继续扩展状态。
- ▶ 使用记忆化搜索枚举这些状态即可。设 $W = \max w_i$ ，时间复杂度为 $\mathcal{O}(nm(\sqrt{W} + 1)^m)$ ，空间复杂度为 $\mathcal{O}(n(\sqrt{W} + 1)^m)$ 。瓶颈是状态数随道路数量指数增长。

数据点 5 ~ 8 40 Pts

- ▶ 有向图不存在环，因此任何道路最多经过一次，问题变为 DAG 上的最长路。

数据点 5 ~ 8 40 Pts

- ▶ 有向图不存在环，因此任何道路最多经过一次，问题变为 DAG 上的最长路。
- ▶ 设 f_v 表示从起点到达驿站 v 时能够取得的最大金线数，初值为 $f_s = 0$ ，其余状态为负无穷。

数据点 5 ~ 8 40 Pts

- ▶ 有向图不存在环，因此任何道路最多经过一次，问题变为 DAG 上的最长路。
- ▶ 设 f_v 表示从起点到达驿站 v 时能够取得的最大金线数，初值为 $f_s = 0$ ，其余状态为负无穷。
- ▶ 按拓扑序处理道路 (u, v, w) ，转移为

$$f_v \leftarrow \max(f_v, f_u + w).$$

最终答案为所有可达状态的最大值，时间复杂度为 $\mathcal{O}(n + m)$ 。

数据点 9 ~ 12 60 Pts

- ▶ 所有驿站互相可达，所以可以从起点到达每一条道路，取完它当前的收益后再回到需要的位置。

数据点 9 ~ 12 60 Pts

- ▶ 所有驿站互相可达，所以可以从起点到达每一条道路，取完它当前的收益后再回到需要的位置。
- ▶ 设 k 是满足 $k(k+1)/2 \leq w$ 的最大整数。一条权值为 w 的道路能够产生的总收益为

$$\sum_{j=0}^k \left(w - \frac{j(j+1)}{2} \right) = (k+1)w - \frac{k(k+1)(k+2)}{6}.$$

数据点 9 ~ 12 60 Pts

- ▶ 所有驿站互相可达，所以可以从起点到达每一条道路，取完它当前的收益后再回到需要的位置。
- ▶ 设 k 是满足 $k(k+1)/2 \leq w$ 的最大整数。一条权值为 w 的道路能够产生的总收益为

$$\sum_{j=0}^k \left(w - \frac{j(j+1)}{2} \right) = (k+1)w - \frac{k(k+1)(k+2)}{6}.$$

- ▶ 将所有道路的这一贡献相加即可。计算 k 时可以二分，所有乘法使用 long long。设 $W = \max w_i$ ，时间复杂度为 $\mathcal{O}(m \log W)$ 。

数据点 13 ~ 15 75 Pts

- ▶ 根据上一档部分分，同一个强连通分量内部的所有道路都可以取得完整收益，并且离开以后没有必要返回。

数据点 13 ~ 15 75 Pts

- ▶ 根据上一档部分分，同一个强连通分量内部的所有道路都可以取得完整收益，并且离开以后没有必要返回。
- ▶ 将每个强连通分量缩成一个点。特殊性质保证缩点后的图是一条链，只需从起点所在分量开始，依次累加分量内部贡献与相邻分量之间最大的道路权值。

数据点 13 ~ 15 75 Pts

- ▶ 根据上一档部分分，同一个强连通分量内部的所有道路都可以取得完整收益，并且离开以后没有必要返回。
- ▶ 将每个强连通分量缩成一个点。特殊性质保证缩点后的图是一条链，只需从起点所在分量开始，依次累加分量内部贡献与相邻分量之间最大的道路权值。
- ▶ 这一档已经得到正解所需的两个部分：强连通分量内部求和，以及缩点后只经过一次的道路贡献。时间复杂度为 $\mathcal{O}(n + m \log W)$ 。

正解 100 Pts

- 使用 Tarjan 求出所有强连通分量。对于分量内部的道路，将前一档得到的完整收益加入分量权值 g_u ；跨分量道路只保留原权值。

正解 100 Pts

- ▶ 使用 Tarjan 求出所有强连通分量。对于分量内部的道路，将前一档得到的完整收益加入分量权值 g_u ；跨分量道路只保留原权值。
- ▶ 缩点后一定得到 DAG。设 f_u 表示从起点出发，到达分量 u 并取完该分量内部收益后，能够取得的最大金线数。

正解 100 Pts

- ▶ 使用 Tarjan 求出所有强连通分量。对于分量内部的道路，将前一档得到的完整收益加入分量权值 g_u ；跨分量道路只保留原权值。
- ▶ 缩点后一定得到 DAG。设 f_u 表示从起点出发，到达分量 u 并取完该分量内部收益后，能够取得的最大金线数。
- ▶ 只有起点所在分量能够直接到达，因此初值为

$$f_{\text{scc}(s)} = g_{\text{scc}(s)},$$

其余状态均为负无穷。

正解 100 Pts

- 按照缩点图的拓扑序处理。对于一条从分量 u 指向分量 v 、原权值为 w 的道路，完整转移为

$$f_v \leftarrow \max(f_v, f_u + w + g_v).$$

正解 100 Pts

- 按照缩点图的拓扑序处理。对于一条从分量 u 指向分量 v 、原权值为 w 的道路，完整转移为

$$f_v \leftarrow \max(f_v, f_u + w + g_v).$$

- 分量 v 的内部收益在首次进入时统一加入。跨分量道路位于 DAG 上，不可能在离开后再次经过。

正解 100 Pts

- 按照缩点图的拓扑序处理。对于一条从分量 u 指向分量 v 、原权值为 w 的道路，完整转移为

$$f_v \leftarrow \max(f_v, f_u + w + g_v).$$

- 分量 v 的内部收益在首次进入时统一加入。跨分量道路位于 DAG 上，不可能在离开后再次经过。
- 最终答案为所有 f_u 的最大值。代码用入度和队列生成拓扑序，Tarjan、缩点和 DAG 递推均为线性时间，总时间复杂度为 $\mathcal{O}(n + m)$ ，空间复杂度为 $\mathcal{O}(n + m)$ 。

正解 100 Pts

```
1 struct edge {
2     int St,Ed,Next,Val;
3 }Edge[2][MAXN<<1];
4
5 void Edge_Add(int St,int Ed,int Val,int Type) {
6     Edge[Type][++Total[Type]]={St,Ed,Head[Type][St],Val};
7     Head[Type][St]=Total[Type];
8 }
```


正解 100 Pts

```
1 void Tarjan(int Now) {
2     Dfn[Now]=Low[Now]=++cnd;
3     S.push(Now);
4     for (int i=Head[0][Now];i;i=Edge[0][i].Next) {
5         int To=Edge[0][i].Ed;
6         if (!Dfn[To]) {
7             Tarjan(To);
8             Low[Now]=min(Low[Now],Low[To]);
9         }
10        else if (!SCC[To]) {
11            Low[Now]=min(Low[Now],Dfn[To]);
12        }
13    }
```

正解 100 Pts

```
1  if (Dfn[Now]==Low[Now]) {
2      cns++;
3      while (S.top()!=Now) {
4          SCC[S.top()]=cns;S.pop();
5      }
6      SCC[S.top()]=cns;S.pop();
7  }
8  }
9
10 long long Calc(int x) {
11     int Bottom=floor(sqrt(2*x*1.0+0.25))-0.5;
12     return 111*x*(Bottom+1)-111*(Bottom+1)*(Bottom+1)*Bottom/6;
13 }
```

正解 100 Pts

```
1 void TopSort() {
2     for (int i=1;i<=cns;i++) {
3         Dp[i]=-(1ll<<60);
4         if (Deg[i]==0) Q.push(i);
5     }
6     Dp[s]=Sum[s];
7     while (Q.size()) {
8         int Now=Q.front(); Q.pop();
9         Ans=max(Ans,Dp[Now]);
10        for (int i=Head[1][Now];i;i=Edge[1][i].Next) {
11            int To=Edge[1][i].Ed;
12            Dp[To]=max(Dp[To],Dp[Now]+Edge[1][i].Val+Sum[To]);
13            Deg[To]--;
14            if (Deg[To]==0) Q.push(To);
15        }
16    }
17    return;
18 }
```

正解 100 Pts

```
1 void Solve() {
2     scanf("%d%d",&n,&m); Ans=0,Total[0]=Total[1]=0,cnd=0,cns=0;
3     for (int i=1;i<=n;i++) Head[0][i]=Head[1][i]=0,Dfn[i]=Low[i]=0,SCC[i]
4         ]=0,Deg[i]=0,Dp[i]=0,Sum[i]=0;
5     for (int i=1;i<=m;i++) {
6         scanf("%d%d%d",&In1,&In2,&In3);
7         Edge_Add(In1,In2,In3,0);
8     }
9     scanf("%d",&s);
10    for (int i=1;i<=n;i++) {
11        if (!Dfn[i]) Tarjan(i);
12    }s=SCC[s];
```

正解 100 Pts

```
1   for (int i=1;i<=m;i++) {
2       int S1=SCC[Edge[0][i].St],S2=SCC[Edge[0][i].Ed],V=Edge[0][i].Val
3       ;
4       if (S1==S2) {
5           Sum[S1]+=Calc(V);
6       }
7       else {
8           Edge_Add(S1,S2,V,1); Deg[S2]++;
9       }
10  }
11  TopSort();
12  printf("%lld\n",Ans);
13 }
```

正解 100 Pts

```
1 int main() {  
2     freopen("goldknot.in", "r", stdin);  
3     freopen("goldknot.out", "w", stdout);  
4     scanf("%d%d", &c, &T);  
5     while (T-->0) Solve();  
6     return 0;  
7 }
```

Contents

T1 鸪天 (skycall)

T2 雨霖铃 (rainbell)

T3 金缕曲 (goldknot)

T4 蝶恋花 (flutter)

吐槽时间

数据点 1 ~ 4 20 Pts

- 直接使用两个数组分别记录当前的盛放值与留芳值。区间加法和区间取最小值逐项修改，修改后令 $b_i \leftarrow \max(b_i, a_i)$ 。

数据点 1 ~ 4 20 Pts

- ▶ 直接使用两个数组分别记录当前的盛放值与留芳值。区间加法和区间取最小值逐项修改，修改后令 $b_i \leftarrow \max(b_i, a_i)$ 。
- ▶ 三类询问也直接扫描区间。单次操作时间复杂度为 $\mathcal{O}(n)$ ，总时间复杂度为 $\mathcal{O}(nq)$ 。

数据点 1 ~ 4 20 Pts

- ▶ 直接使用两个数组分别记录当前的盛放值与留芳值。区间加法和区间取最小值逐项修改，修改后令 $b_i \leftarrow \max(b_i, a_i)$ 。
- ▶ 三类询问也直接扫描区间。单次操作时间复杂度为 $\mathcal{O}(n)$ ，总时间复杂度为 $\mathcal{O}(nq)$ 。
- ▶ 朴素做法中，区间加法容易整体维护，真正的困难是区间取最小值只修改当前较大的元素。

数据点 5 ~ 9 45 Pts

- 不存在区间取最小值时，普通懒标记线段树即可维护区间和与当前最大值。

数据点 5 ~ 9 45 Pts

- ▶ 不存在区间取最小值时，普通懒标记线段树即可维护区间和与当前最大值。
- ▶ 为了维护历史最大值，除了记录区间当前增加量 a ，还要记录这段标记在下传前曾达到的最大前缀增加量 h 。

数据点 5 ~ 9 45 Pts

- ▶ 不存在区间取最小值时，普通懒标记线段树即可维护区间和与当前最大值。
- ▶ 为了维护历史最大值，除了记录区间当前增加量 a ，还要记录这段标记在上传前曾达到的最大前缀增加量 h 。
- ▶ 合并一段新的增加过程 (a', h') 时，组合后的标记为

$$(a + a', \max(h, a + h')).$$

节点的历史最大值用修改前的当前最大值加上 h' 更新。时间复杂度为 $\mathcal{O}((n + q) \log n)$ 。

数据点 10 ~ 15 75 Pts

- 暂时不询问历史最大值，就得到常见的 Segment Tree Beats。每个节点维护最大值 Max、严格次大值 Second、最大值出现次数与区间和。

数据点 10 ~ 15 75 Pts

- ▶ 暂时不询问历史最大值，就得到常见的 Segment Tree Beats。每个节点维护最大值 Max、严格次大值 Second、最大值出现次数与区间和。
- ▶ 对区间执行 $a_i \leftarrow \min(a_i, v)$ 时，若 $\text{Second} < v < \text{Max}$ ，只有等于最大值的元素会变化，可以整段修改。

数据点 10 ~ 15 75 Pts

- ▶ 暂时不询问历史最大值，就得到常见的 Segment Tree Beats。每个节点维护最大值 Max、严格次大值 Second、最大值出现次数与区间和。
- ▶ 对区间执行 $a_i \leftarrow \min(a_i, v)$ 时，若 $\text{Second} < v < \text{Max}$ ，只有等于最大值的元素会变化，可以整段修改。
- ▶ 每次无法整段处理才继续递归。节点最大值被严格降低后，才可能再次产生这类递归，因此总复杂度为 $\mathcal{O}((n + q) \log^2 n)$ 。

正解 100 Pts

- ▶ 结构体 Data 维护区间长度 Len、区间和 Sum、最大值 Max、严格次大值 SecMax、最大值个数 MaxCnt 与区间历史最大值 HisMax。

正解 100 Pts

- ▶ 结构体 Data 维护区间长度 Len、区间和 Sum、最大值 Max、严格次大值 SecMax、最大值个数 MaxCnt 与区间历史最大值 HisMax。
- ▶ 合并左右儿子时，Sum 与 Len 直接相加，HisMax 取最大值。比较两个儿子的 Max，即可确定新区间的 Max、MaxCnt 与 SecMax。

正解 100 Pts

- ▶ 结构体 Data 维护区间长度 Len、区间和 Sum、最大值 Max、严格次大值 SecMax、最大值个数 MaxCnt 与区间历史最大值 HisMax。
- ▶ 合并左右儿子时，Sum 与 Len 直接相加，HisMax 取最大值。比较两个儿子的 Max，即可确定新区间的 Max、MaxCnt 与 SecMax。
- ▶ 这些信息既能回答三类询问，也能判断一次区间取最小值是否只会改变当前最大值。

正解 100 Pts

- ▶ 结构体 tag 把区间中的元素分成“当前最大值”和“其余元素”两组。MaxAdd、OtherAdd 表示两组最终累计的增加量。

正解 100 Pts

- ▶ 结构体 tag 把区间中的元素分成“当前最大值”和“其余元素”两组。MaxAdd、OtherAdd 表示两组最终累计的增加量。
- ▶ MaxHis、OtherHis 表示两组在整段修改过程中达到过的最大前缀增加量。它们记录的是过程中的峰值，而不只是最后留下的增加量。

正解 100 Pts

- ▶ 结构体 tag 把区间中的元素分成“当前最大值”和“其余元素”两组。MaxAdd、OtherAdd 表示两组最终累计的增加量。
- ▶ MaxHis、OtherHis 表示两组在整段修改过程中达到过的最大前缀增加量。它们记录的是过程中的峰值，而不只是最后留下的增加量。
- ▶ 标记作用于节点时，有

$$\begin{aligned}\text{HisMax}' &= \max(\text{HisMax}, \text{Max} + \text{MaxHis}), \\ \text{Sum}' &= \text{Sum} + \text{MaxAdd} \cdot \text{MaxCnt} \\ &\quad + \text{OtherAdd} \cdot (\text{Len} - \text{MaxCnt}).\end{aligned}$$

正解 100 Pts

- ▶ 结构体 tag 把区间中的元素分成“当前最大值”和“其余元素”两组。MaxAdd、OtherAdd 表示两组最终累计的增加量。
- ▶ MaxHis、OtherHis 表示两组在整段修改过程中达到过的最大前缀增加量。它们记录的是过程中的峰值，而不只是最后留下的增加量。
- ▶ 标记作用于节点时，有

$$\begin{aligned}\text{HisMax}' &= \max(\text{HisMax}, \text{Max} + \text{MaxHis}), \\ \text{Sum}' &= \text{Sum} + \text{MaxAdd} \cdot \text{MaxCnt} \\ &\quad + \text{OtherAdd} \cdot (\text{Len} - \text{MaxCnt}).\end{aligned}$$

- ▶ 随后令 $\text{Max} \leftarrow \text{Max} + \text{MaxAdd}$ ，并令有效的 $\text{SecMax} \leftarrow \text{SecMax} + \text{OtherAdd}$ 。

正解 100 Pts

- 规定 $A \circ B$ 表示先执行标记 B ，再执行标记 A 。以最大值组为例，复合后的标记满足

$$\text{MaxAdd}_{A \circ B} = \text{MaxAdd}_B + \text{MaxAdd}_A,$$

$$\text{MaxHis}_{A \circ B} = \max(\text{MaxHis}_B, \text{MaxAdd}_B + \text{MaxHis}_A).$$

正解 100 Pts

- 规定 $A \circ B$ 表示先执行标记 B ，再执行标记 A 。以最大值组为例，复合后的标记满足

$$\text{MaxAdd}_{A \circ B} = \text{MaxAdd}_B + \text{MaxAdd}_A,$$

$$\text{MaxHis}_{A \circ B} = \max(\text{MaxHis}_B, \text{MaxAdd}_B + \text{MaxHis}_A).$$

- 其余元素组完全相同，只需将上式中的 Max 换成 Other 。

正解 100 Pts

- 规定 $A \circ B$ 表示先执行标记 B ，再执行标记 A 。以最大值组为例，复合后的标记满足

$$\text{MaxAdd}_{A \circ B} = \text{MaxAdd}_B + \text{MaxAdd}_A,$$

$$\text{MaxHis}_{A \circ B} = \max(\text{MaxHis}_B, \text{MaxAdd}_B + \text{MaxHis}_A).$$

- 其余元素组完全相同，只需将上式中的 Max 换成 Other 。
- 因此 $\text{Mark}(\text{Now}, \text{Tag})$ 先用新标记修改节点信息，再令

$$\text{Node}[\text{Now}].\text{Tag} \leftarrow \text{Tag} \circ \text{Node}[\text{Now}].\text{Tag}.$$

代码使用 `operator*` 实现这一复合，这与修改发生的先后顺序一致。

正解 100 Pts

- ▶ 区间加 k 时，两组元素都增加 k ，对应标记为 (k, k, k, k) 。

正解 100 Pts

- ▶ 区间加 k 时，两组元素都增加 k ，对应标记为 (k, k, k, k) 。
- ▶ 执行区间取最小值时，若 $\text{SecMax} < v < \text{Max}$ ，只有当前最大值会降低。令 $d = v - \text{Max} < 0$ ，对应标记为 $(d, 0, 0, 0)$ 。这次操作不会产生新的历史最大值，所以两个历史前缀量均为 0。

正解 100 Pts

- ▶ 区间加 k 时，两组元素都增加 k ，对应标记为 (k, k, k, k) 。
- ▶ 执行区间取最小值时，若 $\text{SecMax} < v < \text{Max}$ ，只有当前最大值会降低。令 $d = v - \text{Max} < 0$ ，对应标记为 $(d, 0, 0, 0)$ 。这次操作不会产生新的历史最大值，所以两个历史前缀量均为 0。
- ▶ 下传时，最大值等于父亲最大值的儿子接收完整标记。另一个儿子的所有元素都属于父亲的其余元素组，因此接收

$(\text{OtherAdd}, \text{OtherAdd}, \text{OtherHis}, \text{OtherHis})$.

正解 100 Pts

- 无法整段执行区间取最小值时继续递归。每次进入递归都会使某个节点的最大值严格降低，沿用 Segment Tree Beats 的势能分析。

正解 100 Pts

- ▶ 无法整段执行区间取最小值时继续递归。每次进入递归都会使某个节点的最大值严格降低，沿用 Segment Tree Beats 的势能分析。
- ▶ 区间和必须使用 long long。本题权值始终在 int 范围内，SecMax 的无效值设为足够小的负无穷。

正解 100 Pts

- ▶ 无法整段执行区间取最小值时继续递归。每次进入递归都会使某个节点的最大值严格降低，沿用 Segment Tree Beats 的势能分析。
- ▶ 区间和必须使用 long long。本题权值始终在 int 范围内，SecMax 的无效值设为足够小的负无穷。
- ▶ 时间复杂度为 $\mathcal{O}((n+q)\log^2 n)$ ，空间复杂度为 $\mathcal{O}(n)$ 。

正解 100 Pts

```
1
2 int n,m,In1,In2,In3,cnt=0,Line[MAXN];
3
4 struct Data{
5     int Len,Max,SecMax,HisMax,MaxCnt;
6     long long Sum;
7 };
8
9 struct tag{
10     int MaxAdd,OtherAdd,MaxHis,OtherHis;
11 };
12
13 tag Empty(){return {0,0,0,0};}
14 Data Null(){return {0,-INF,-INF,-INF,0,0};}
```

正解 100 Pts

```
1 Data operator+(const Data &A,const Data &B){
2     if(!A.Len) return B;
3     if(!B.Len) return A;
4     Data Ret={A.Len+B.Len,0,0,max(A.HisMax,B.HisMax),0,A.Sum+B.Sum};
5     if(A.Max==B.Max) Ret.Max=A.Max,Ret.MaxCnt=A.MaxCnt+B.MaxCnt,Ret.
        SecMax=max(A.SecMax,B.SecMax);
6     else if(A.Max>B.Max) Ret.Max=A.Max,Ret.MaxCnt=A.MaxCnt,Ret.SecMax=
        max(A.SecMax,B.Max);
7     else Ret.Max=B.Max,Ret.MaxCnt=B.MaxCnt,Ret.SecMax=max(A.Max,B.SecMax
        );
8     return Ret;
9 }
```

正解 100 Pts

```
1 Data operator*(const tag &Tag,const Data &B){
2     Data Ret=B;
3     Ret.HisMax=max(Ret.HisMax,Ret.Max+Tag.MaxHis);
4     Ret.Sum+=1ll*Tag.MaxAdd*Ret.MaxCnt+1ll*Tag.OtherAdd*(Ret.Len-Ret.
        MaxCnt);
5     Ret.Max+=Tag.MaxAdd;
6     if(Ret.SecMax!=-INF) Ret.SecMax+=Tag.OtherAdd;
7     return Ret;
8 }
9
10 tag operator*(const tag &A,const tag &B){
11     tag Ret=B;
12     Ret.MaxHis=max(Ret.MaxHis,Ret.MaxAdd+A.MaxHis);
13     Ret.OtherHis=max(Ret.OtherHis,Ret.OtherAdd+A.OtherHis);
14     Ret.MaxAdd+=A.MaxAdd;Ret.OtherAdd+=A.OtherAdd;
15     return Ret;
16 }
```

正解 100 Pts

```
1 struct node{
2     int l,r,LeftSon,RightSon;
3     Data Val;
4     tag Tag;
5 }Node[MAXN<<2];
6
7 void Update(int Now){Node[Now].Val=Node[LSon].Val+Node[RSon].Val;}
8
9 void Mark(int Now,tag Tag){
10     Node[Now].Val=Tag*Node[Now].Val;
11     Node[Now].Tag=Tag*Node[Now].Tag;
12     return;
13 }
```

正解 100 Pts

```
1 void PushDown(int Now){
2     tag Tag=Node[Now].Tag;
3     if(!Tag.MaxAdd&&!Tag.OtherAdd&&!Tag.MaxHis&&!Tag.OtherHis) return;
4     int Max=max(Node[Lson].Val.Max,Node[Rson].Val.Max);
5     tag Other={Tag.OtherAdd,Tag.OtherAdd,Tag.OtherHis,Tag.OtherHis};
6     if(Node[Lson].Val.Max==Max) Mark(Lson,Tag);else Mark(Lson,Other);
7     if(Node[Rson].Val.Max==Max) Mark(Rson,Tag);else Mark(Rson,Other);
8     Node[Now].Tag=Empty();
9     return;
10 }
```

正解 100 Pts

```
1 int Build(int l,int r){
2     int Now=++cnt;
3     Node[Now]={l,r,0,0,{r-l+1,-INF,-INF,-INF,0,0},Empty()};
4     if(l==r){Node[Now].Val={1,Line[1],-INF,Line[1],1,Line[1]};return Now
5         ;}
6     int Mid=(l+r)>>1;
7     LSon=Build(l,Mid);
8     RSon=Build(Mid+1,r);
9     Update(Now);
10    return Now;
11 }
```

正解 100 Pts

```
1 void Modify(int Now,int l,int r,tag Tag){
2     if(Node[Now].l>=l&&Node[Now].r<=r){Mark(Now,Tag);return;}
3     else if(Node[Now].l>r||Node[Now].r<l) return;
4     PushDown(Now);
5     Modify(LSon,l,r,Tag);Modify(RSon,l,r,Tag);
6     Update(Now);
7     return;
8 }
```

正解 100 Pts

```
1 void Limit(int Now,int l,int r,int Val){
2     if(Node[Now].l>r||Node[Now].r<l||Node[Now].Val.Max<=Val) return;
3     if(Node[Now].l>=l&&Node[Now].r<=r&&Node[Now].Val.SecMax<Val){
4         Mark(Now,{Val-Node[Now].Val.Max,0,0,0});
5         return;
6     }
7     PushDown(Now);
8     Limit(LSon,l,r,Val);Limit(RSon,l,r,Val);
9     Update(Now);
10    return;
11 }
```


正解 100 Pts

```
1 Data Query(int Now,int l,int r){
2     if(Node[Now].l>=l&&Node[Now].r<=r) return Node[Now].Val;
3     else if(Node[Now].l>r||Node[Now].r<l) return Null();
4     PushDown(Now);
5     return Query(LSon,l,r)+Query(RSon,l,r);
6 }
```

正解 100 Pts

```
1  int main(){
2      freopen("flutter.in", "r", stdin);
3      freopen("flutter.out", "w", stdout);
4      int c, T;
5      scanf("%d%d", &c, &T);
6      while(T-->0){
7          scanf("%d%d", &n, &m);
8          cnt=0;
9          for(int i=1; i<=n; i++) scanf("%d", &Line[i]);
10         Build(1, n);
```

正解 100 Pts

```
1      while(m-->0){
2          scanf("%d%d%d",&In1,&In2,&In3);
3          if(In1==1){scanf("%d",&In1);Modify(1,In2,In3,{In1,In1,In1,
In1});}
4          else if(In1==2){scanf("%d",&In1);Limit(1,In2,In3,In1);}
5          else if(In1==3) printf("%lld\n",Query(1,In2,In3).Sum);
6          else if(In1==4) printf("%d\n",Query(1,In2,In3).Max);
7          else printf("%d\n",Query(1,In2,In3).HisMax);
8      }
9  }
10 return 0;
11 }
```

Contents

T1 鸪天 (skycall)

T2 雨霖铃 (rainbell)

T3 金缕曲 (goldknot)

T4 蝶恋花 (flutter)

吐槽时间